Computer Architecture Lab

(EL-2012)

LABORATORY REPORT

Fall 2026



(Lab#09)

Implementation of a Register File using

ModelSim

Student Name: Manahil Shahzad **Roll No:** 24I-6563 **Section:** A

Date performed:**27th Oct,2026**
MARKS AWARDED: _____ / 10

Signature and date of Instructor: _____
Lab Instructors: Dr. Nasir Ali Shah, Engr. Fasih Ahmad

Task 01+Task 02: Write Verilog Code for 4x4 Register file and Simulate the register file for following conditions;

i. Write 6 in register1 and 3 to register2.
ii. Write 13 to register 3
iii. Read register 1 & 2 on out1 & out2, respectively.

Description:

In Task 1, we design a 4×4 register file using Verilog, which contains four registers of 4-bit size. In Task 2, we simulate the register file in ModelSim by writing 6 into Register 1, 3 into Register 2, and 13 into Register 3. Finally, we read Register 1 and Register 2 and check their outputs.

Code:

```
module decoder_2_to4(output reg [3:0] dec_out, input [1:0] wr, input write_enable);
   always @(*) begin
      if (write_enable) begin
         if (wr == 2'b00)
            dec_out = 4'b0001;
         else if (wr == 2'b01)
            dec_out = 4'b0010;
         else if (wr == 2'b10)
            dec_out = 4'b0100;
         else if (wr == 2'b11)
            dec_out = 4'b1000;
         else
            dec_out = 4'b0000;
      end
      else
         dec_out = 4'b0000;
   end
endmodule

module mux4_1(output reg [3:0] out, input [3:0] in0, in1, in2, in3, input [1:0] sel);
   always @(*) begin
      if (sel == 2'b00)
         out = in0;
      else if (sel == 2'b01)
         out = in1;
      else if (sel == 2'b10)
         out = in2;
      else if (sel == 2'b11)
         out = in3;
      else
         out = 4'b0000;
   end
```

```
endmodule

module register_4bit(output reg [3:0] q, input [3:0] d, input load, input clk);
   always @(posedge clk) begin
      if (load)
         q <= d;
   end
endmodule

module register_file_4x4(data_out1, data_out2, data_in, wr, write_enable, rd1, rd2, clk);
   output [3:0] data_out1, data_out2;
   input [3:0] data_in;
   input [1:0] wr, rd1, rd2;
   input write_enable, clk;
   wire [3:0] dec_out, q0, q1, q2, q3;

   decoder_2_to4 decoder(dec_out, wr, write_enable);

   register_4bit reg0(q0, data_in, dec_out[0], clk);
   register_4bit reg1(q1, data_in, dec_out[1], clk);
   register_4bit reg2(q2, data_in, dec_out[2], clk);
   register_4bit reg3(q3, data_in, dec_out[3], clk);

   mux4_1 mux1(data_out1, q0, q1, q2, q3, rd1);
   mux4_1 mux2(data_out2, q0, q1, q2, q3, rd2);
endmodule

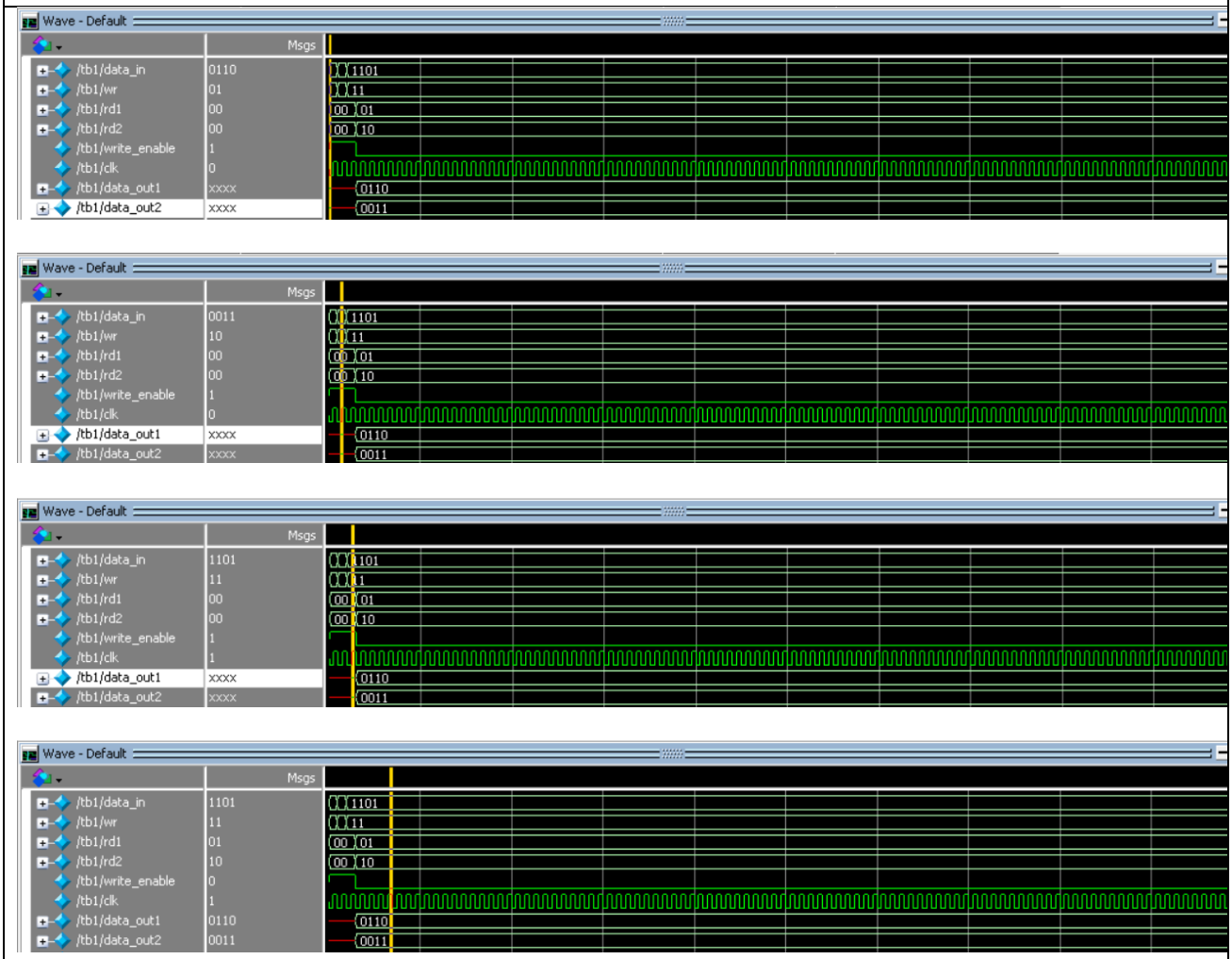
module tb1;
   reg [3:0] data_in;
   reg [1:0] wr, rd1, rd2;
   reg write_enable, clk;
   wire [3:0] data_out1, data_out2;
   register_file_4x4 reg_file(data_out1, data_out2, data_in, wr, write_enable, rd1, rd2, clk);
   initial begin
      clk = 0;
      forever #5 clk = ~clk;
   end
   initial begin
      write_enable = 0;
      data_in = 0;
      wr = 0;

      rd1 = 0;
      rd2 = 0;

      write_enable = 1;
      wr = 2'b01;
```

```
        data_in = 4'b0110;
        #10;
        wr = 2'b10;
        data_in = 4'b0011;
        #10;
        wr = 2'b11;
        data_in = 4'b1101;
        #10;
        write_enable = 0;
        rd1 = 2'b01;
        rd2 = 2'b10;
        #10;
    end
endmodule
```

Output:



Task 03: Design, implement, and verify a 32-bit Register File for a RISC-V (RV32) processor architecture using Verilog.

Description:

In this task, we design a 32-bit register file for a RISC-V processor using Verilog. It contains 32 registers, each of 32-bit size, with two read ports and one write port. We simulate the circuit to check its reading, writing, and resetting operations.

Code:

```
module regfile_rv32(clk, rst, we, rs1, rs2, rd, wdata, rdata1, rdata2);
   input clk, rst, we;
   input [4:0] rs1, rs2, rd;
   input [31:0] wdata;
   output [31:0] rdata1, rdata2;

   reg [31:0] regs [0:31];
   integer i;

   always @(posedge clk) begin
      if (rst) begin
         for (i = 0; i < 32; i = i + 1)
            regs[i] <= 32'd0;
      end
      else if (we && rd != 5'd0)
         regs[rd] <= wdata;
   end

   assign rdata1 = (rs1 == 5'd0) ? 32'd0 : regs[rs1];
   assign rdata2 = (rs2 == 5'd0) ? 32'd0 : regs[rs2];
endmodule



module tb_task3;
   reg clk, rst, we;
   reg [4:0] rs1, rs2, rd;
   reg [31:0] wdata;
   wire [31:0] rdata1, rdata2;

   regfile_rv32 rf(clk, rst, we, rs1, rs2, rd, wdata, rdata1, rdata2);

   initial begin
      clk = 0;
      forever #5 clk = ~clk;
   end
```

```
initial begin
    we = 0;
    rs1 = 0;
    rs2 = 0;
    rd = 0;
    wdata = 0;


    rst = 1;
    #10;
    rst = 0;


    we = 1;
    rd = 5'd5;
    wdata = 32'hDEADBEEF;
    #10;

    rd = 5'd6;
    wdata = 32'h12345678;
    #10;


    we = 0;
    rs1 = 5'd5;
    rs2 = 5'd6;
    #10;

    rs1 = 5'd6;
    rs2 = 5'd5;
    #10;

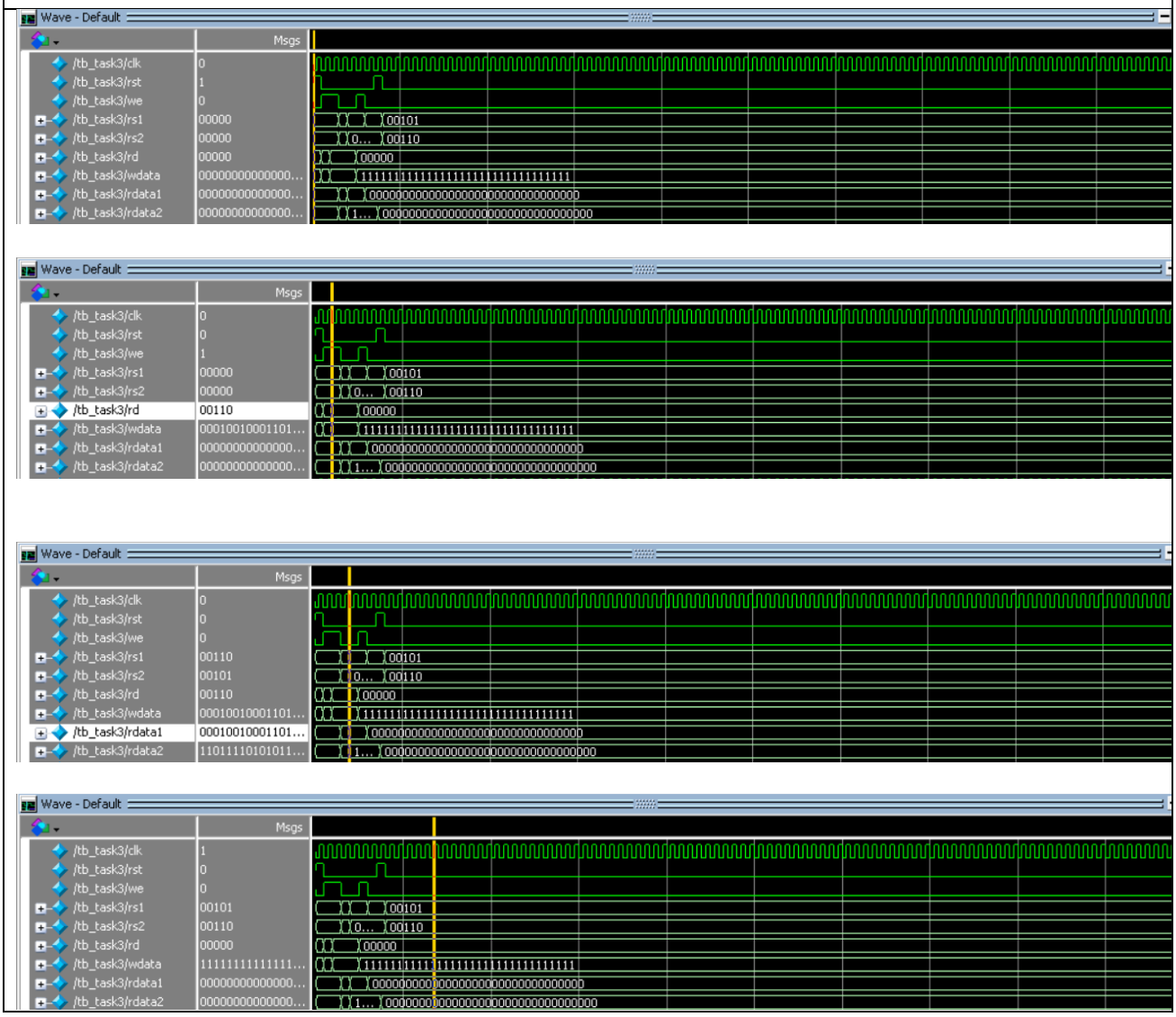

    we = 1;
    rd = 5'd0;
    wdata = 32'hFFFFFFFF;
    #10;
    we = 0;
    rs1 = 5'd0;
    #10;


    rst = 1;
    #10;
    rst = 0;
    rs1 = 5'd5;
```

```
        rs2 = 5'd6;
        #10;

    end
endmodule
```

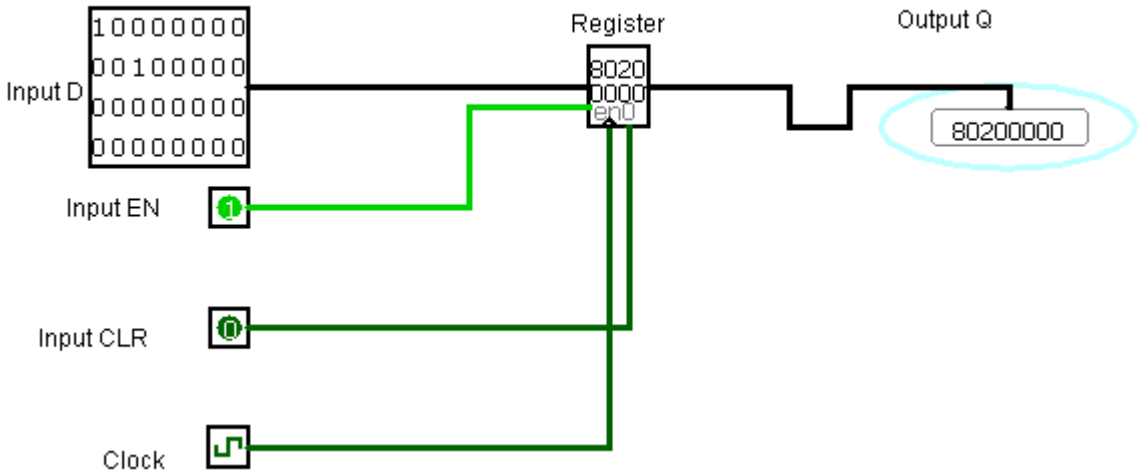
Output:



Task 04: Simulate a register using logisim.

Description:
In this task, we design a 32-bit register using Logisim. We apply binary data at the input and use a clock to store the data. The output shows the stored data and keeps it until the next clock edge.

Code:



Output:

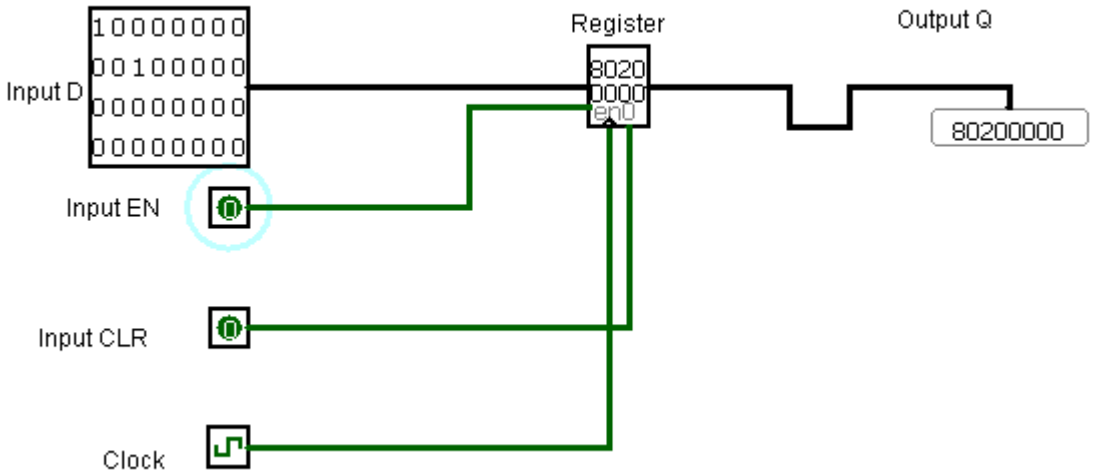


Figure 1 EN=0, so Q keeps its old value even when D changes

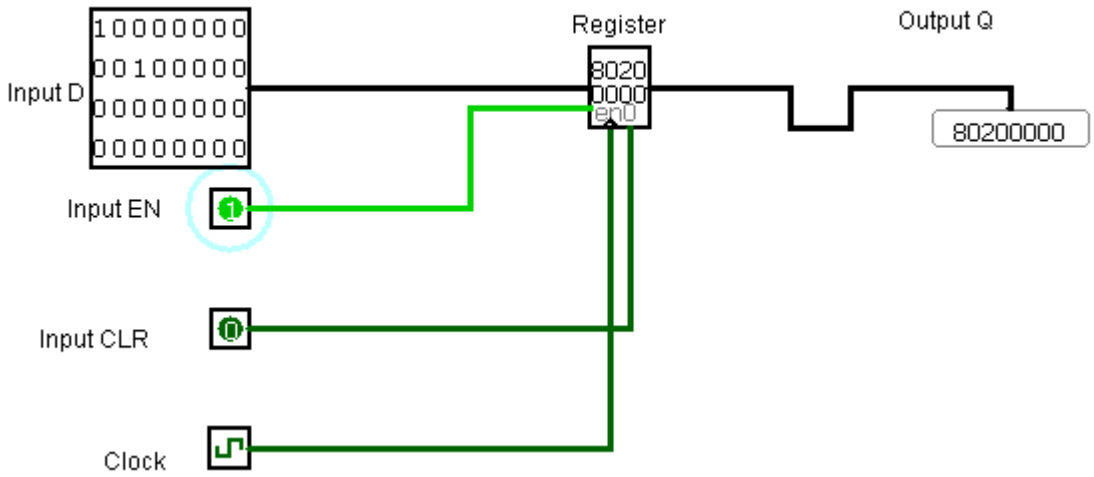


Figure 2 EN=1, so Q takes the value of D on the rising edge

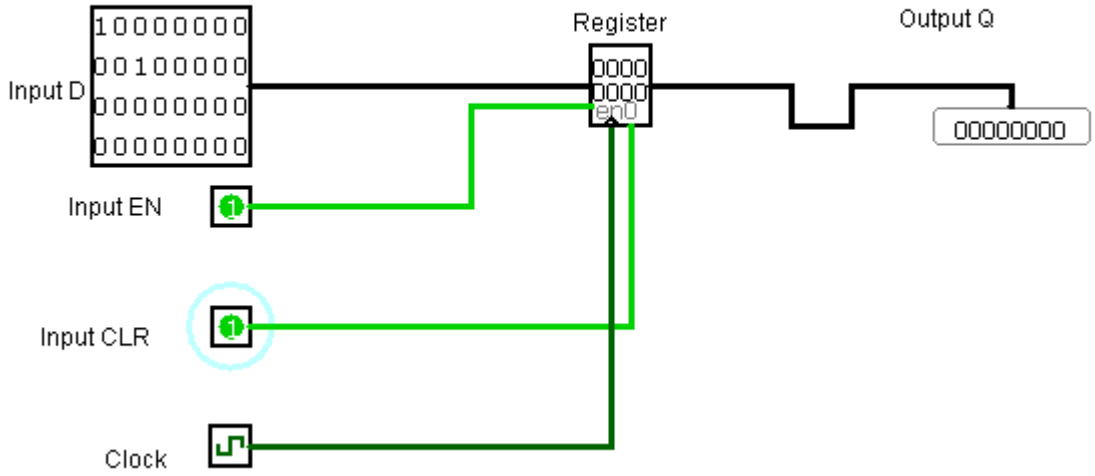


Figure 3 CLR=1, so Q resets to 0